

CODEGUARDIAN AI OUTPUT SCREENSHOT

Deploy ⋮

CodeGuardian AI

Secure • Detect • Analyze Your Code

Select

☒ Login

☐ Sign Up

Username

Anirudh

Password

Login

Deploy ⋮

Anirudh

Navigation

☒ Upload & Scan

☐ Analysis

☐ Visualization

☐ Logout

CodeGuardian AI

Secure • Detect • Analyze Your Code

 Upload Your Project

Upload file or ZIP project

 Drag and drop file here

Limit 200MB per file • ZIP, PY, JS, JAVA, C, CPP

Browse files

Deploy ⋮

Anirudh Deepak

Navigation

☒ Upload & Scan

☐ Analysis

☐ Visualization

☐ Logout

CodeGuardian AI

Secure • Detect • Analyze Your Code

 Upload Your Project

Upload file or ZIP project

 Drag and drop file here

Limit 200MB per file • ZIP, PY, JS, JAVA, C, CPP

Browse files

CodeGuardian AI

Secure • Detect • Analyze Your Code

Upload Your Project

Upload file or ZIP project

 Drag and drop file here
Limit 200MB per file • ZIP, PY, JS, JAVA, C, CPP

Browse files

 file.py 0.7KB

×

 Run Scan

✓ Scan Completed!

Analysis

 Most Vulnerable File: uploaded_project\file.py

Security Findings Summary

⚠️ 2 Critical issue(s) require immediate attention

Total	Critical	High	Medium	Low
9	2	4	1	2

uploaded_project\file.py (Line 2)

```
AWS Access Key = AK*****LE
```

Severity: CRITICAL | Score: 10

Occurrences: 1

Why Dangerous:

AWS Access Keys grant direct programmatic access to AWS resources. If exposed, an attacker can impersonate the user or role associated with the key, leading to unauthorized data access, modification, or deletion, and incurring costs.

Fix:

Do not embed AWS credentials directly in code. Use IAM roles for EC2 instances or other AWS services. For local development or applications not running on AWS, use the AWS CLI to configure credentials, or environment variables, and leverage libraries that automatically pick them up securely.

uploaded_project\file.py (Line 3)

```
Private Key = -----
```

Severity: CRITICAL | Score: 10

Occurrences: 1

Why Dangerous:

Exposing private keys in code allows unauthorized access to encrypted data or systems.

Fix:

Store private keys securely outside of your codebase. Use environment variables, a secrets management system (like HashiCorp Vault, AWS Secrets Manager, Azure Key Vault), or encrypted configuration files. Load the key at runtime, not hardcoded.

uploaded_project\file.py (Line 6)

```
API Key = sk*****EY
```

Severity: HIGH | Score: 8

Occurrences: 1

Why Dangerous:

Exposing API keys directly in code makes them vulnerable to unauthorized access and misuse, potentially leading to account compromise and data breaches.

Fix:

Store API keys in environment variables or a dedicated secrets management system and access them programmatically. For example, in Python: `import os`
`API_KEY = os.environ.get("MY_APP_API_KEY")`

uploaded_project\file.py (Line 7)

```
API Key = pk*****OP
```

Severity: HIGH | Score: 8

Occurrences: 1

Why Dangerous:

Exposing API keys in code allows unauthorized access to services and data, leading to potential misuse, data breaches, or financial losses.

Fix:

Store the API key in environment variables and access it programmatically. For example: `import os client_key = os.environ.get("STRIPE_API_KEY")` Ensure this environment variable is set securely in your deployment environment and not committed to version control.

uploaded_project\file.py (Line 11)

```
Access Token = gh*****56
```

Severity: HIGH | Score: 8

Occurrences: 1

Why Dangerous:

Exposes a GitHub Personal Access Token, which can be used to access and control the associated GitHub account and its repositories.

Fix:

Remove the token from the code. Use environment variables or a secrets management tool for storing and accessing sensitive credentials. For example, access the token via `os.environ.get("GITHUB_TOKEN")` after setting it as an environment variable.

uploaded_project\file.py (Line 18)

```
API Key = sk*****ET
```

Severity: HIGH | Score: 8

Occurrences: 1

Why Dangerous:

API keys grant access to services and sensitive data. Exposing them in code makes them vulnerable to unauthorized access and misuse.

Fix:

Do not store API keys directly in code. Use environment variables or a secrets management system to store and access them securely. For example, set an environment variable named `API_KEY` with the value `"sk-duplicate123SECRET"` and access it in your code using `os.environ.get('API_KEY')`.

uploaded_project\file.py (Line 10)

```
Password = My*****23
```

Severity: MEDIUM | Score: 5

Occurrences: 1

Why Dangerous:

Plaintext passwords in code are a severe security vulnerability. Anyone with access to the codebase can see the password, compromising the system it protects.

Fix:

Store credentials securely using environment variables or a dedicated secrets management system. Do not hardcode them. For example, instead of `password = "MySecurePassword123"`, use `password = os.environ.get("DATABASE_PASSWORD")`.

uploaded_project\file.py (Line 14)

```
Potential Secret = x7*****B1
```

Severity: LOW | Score: 3

Occurrences: 1

Why Dangerous:

Hardcoded credentials like this are easily discoverable by attackers who gain access to the codebase. This can lead to unauthorized access to systems or data.

Fix:

Store secrets in a secure secrets management system (e.g., AWS Secrets Manager, HashiCorp Vault, Azure Key Vault) and retrieve them at runtime. Alternatively, use environment variables that are not checked into version control.

uploaded_project\file.py (Line 15)

```
Potential Secret = Zx*****LM
```

Severity: LOW | Score: 3

Occurrences: 1

Why Dangerous:

Hardcoded secrets like this are extremely risky. If the code is leaked, accessed by unauthorized individuals, or even if an employee with access mistakenly shares it, the secret is compromised. This could lead to unauthorized access to sensitive systems or data.

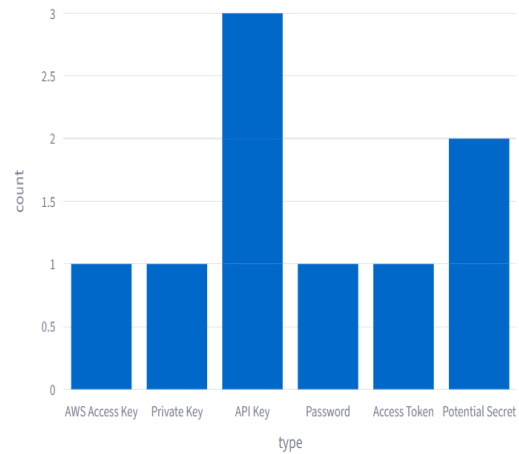
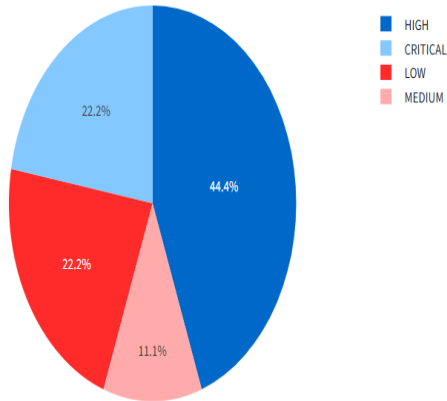
Fix:

Do not hardcode secrets. Use environment variables or a secrets management system (like HashiCorp Vault, AWS Secrets Manager, Azure Key Vault) to store and retrieve secrets securely. For example, if using environment variables:

```
another_secret = os.environ.get("ANOTHER_SECRET")
```



Visualization



Anirudh Deepak

Navigation

- ☐ Upload & Scan
- ☒ Analysis
- ☐ Visualization
- ☐ Logout

Deploy

Severity: LOW | Score: 3

Occurrences: 1

Why Dangerous:

Hardcoded secrets like this are extremely risky. If the code is leaked, accessed by unauthorized individuals, or even if an employee with access mistakenly shares it, the secret is compromised. This could lead to unauthorized access to sensitive systems or data.

Fix:

Do not hardcode secrets. Use environment variables or a secrets management system (like HashiCorp Vault, AWS Secrets Manager, Azure Key Vault) to store and retrieve secrets securely. For example, if using environment variables:

```
another_secret = os.environ.get("ANOTHER_SECRET")
```

Download Report



report.csv
5.4 KB • Done

file	line	type	code	value	raw_value	occurrence	severity	recommender	danger	fix	score
uploaded_	2	AWS Access Key	aws_key = AKI****PL	AKIAIOSF		1	CRITICAL	Use IAM role for AWS Access	Do not embed AWS credentials in code		10
uploaded_	3	Private Key	private_key = -----BEGIN			1	CRITICAL	Never expose private keys	Store private keys securely		10
uploaded_	6	API Key	API_KEY = sk-****KE	sk-abc123		1	HIGH	Store API keys in a secure	Store API keys in a secure		8
uploaded_	7	API Key	client_key = pk_****N	pk_live_51		1	HIGH	Store API keys in a secure	Store the API key in a secure		8
uploaded_	11	Access Token	token = "ghp****45"	ghp_12345		1	HIGH	Tokens should be removed	Remove the token from the code		8
uploaded_	18	API Key	API_KEY = sk-****RE	sk-duplicat		1	HIGH	Store API keys in a secure	Do not store API keys directly in code		8
uploaded_	10	Password	password = MyS*****1	MySecureP		1	MEDIUM	Avoid hardcoding passwords	Store credentials securely		5
uploaded_	14	Potential Secret	random_secret = x7F****vE	x7F9kL2pC		1	LOW	Review this for hardcoding	Store secrets in a secure		3
uploaded_	15	Potential Secret	another_secret = ZxY****KL	ZxY987654		1	LOW	Review this for hardcoding	Do not hardcode secrets		3

By: Anirudh Deepak, Aswanth A A, ASWIN S